Харківський національний університет радіоелектроніки
Факультет комп’ютерних наук
Кафедра програмної інженерії

ЗВІТ

з лабораторної роботи № 1
з дисципліни «Програмування мовою Dart»
на тему: «Функції, замикання, колекції, рядки та дати»
Розклад занять групи (варіант 4)

Виконала:
ст. гр. ПЗПІ-23-9
Курінна Д. С.

Перевірив:
асистент каф. ПІ
Баличев М. Д.

Харків 2026

1.1 Мета роботи та завдання

Мета роботи: набути практичних навичок розв’язання задач мовою Dart із функціями, замиканнями, колекціями, рядками, регулярними виразами та датами без використання класів.

Завдання за варіантом 4.

Предметна область – «Розклад занять групи», основна сутність – заняття з полями: дисципліна, викладач, тип, день тижня, час початку, аудиторія. До них додано сьоме поле – дату першого заняття в цьому слоті розкладу, оскільки завдання передбачає роботу з датами. Вхідні дані – розклад моєї групи (ПЗПІ-23-9) на осінній семестр 2026/2027 навчального року. Усі заняття дистанційні, тому в полі аудиторії записано `DL`.

Програма має:

1) зберігати вхідні дані як константний список щонайменше з 15 рядків, серед яких не менше трьох пошкоджених;

2) розбирати рядки без аварійного завершення за допомогою `int.tryParse` і `DateTime.tryParse`;

3) перевіряти значення на допустимі діапазони й повертати перелік порушень;

4) виконувати конвеєри фільтрації, перетворення та згортання колекцій;

5) групувати заняття в `Map<String, List<…>>` і рахувати агрегат для кожної групи;

6) сортувати за двома критеріями з іменованим параметром напряму;

7) формувати текстовий звіт через `StringBuffer` з іменованими параметрами;

8) використовувати замикання;

9) застосовувати регулярний вираз з іменованими групами та `replaceAllMapped`;

10) обчислювати вік запису в днях через `DateTime.difference`.

1.2 Хід роботи

1.2.1 Структура програми

Проєкт створено командою `dart create -t console lab1_schedule`. Код розділено між `bin/` і `lib/` (таблиця 1). Уся логіка лежить у `lib/`, а в `bin/` лишається сценарій запуску. Так функції можна підключати з модульних тестів у наступних роботах, а точка входу не залежить від їхньої реалізації. Альтернативою був один файл у `bin/`, але в ньому важко відокремити логіку від виводу, і його не можна імпортувати.

Зовнішніх залежностей часу виконання немає. Для аналізу підключено dev-залежність `lints`: набір правил `recommended` підсилено режимами `strict-casts`, `strict-inference` і `strict-raw-types`. Повний код доступний у репозиторії https://github.com/NureKurinnaDaria/dart-lab1-schedule (гілка `lab1`).

Таблиця 1 – Склад проєкту

Файл	Призначення
`bin/lab1_schedule.dart`	Точка входу: виклики функцій і вивід
`lib/data.dart`	Константний список із 22 вхідних рядків
`lib/lesson.dart`	Тип заняття, розбір і валідація
`lib/analysis.dart`	Замикання, конвеєри, групування, сортування, дати
`lib/report.dart`	Маскування прізвищ і формування звіту

1.2.2 Модель даних

Заняття описано записом Dart (record) через `typedef` (лістинг 1). `Map<String, Object>` не перевіряє типи під час компіляції, а кортеж без імен провокує плутанину позицій, тому обрано запис: він дає типізовані іменовані поля без класу. Час початку зберігається як `int` (хвилини від початку доби), бо його легко порівнювати, сортувати й усереднювати; `DateTime` не підходить, оскільки завжди містить дату.

Результат розбору – запис `ParseResult`: або заняття, або текст помилки. Помилка вхідного рядка є очікуваною ситуацією, тому замість винятку повертається значення.

Лістинг 1 – Типи даних

```
typedef Lesson = ({
  String subject,
  String teacher,
  String type,
  int weekday,
  int startMinutes,
  String room,
  DateTime firstDate,
});

/// Результат розбору рядка: або заняття, або опис помилки.
typedef ParseResult = ({Lesson? lesson, String? error});
```

1.2.3 Розбір і перевірка рядків

Функція `parseLine` ділить рядок за роздільником, перевіряє кількість полів і перетворює числові значення через `tryParse` (лістинг 2). Вона не кидає винятків, тому в програмі немає `try/catch`.

Лістинг 2 – Розбір рядка (скорочено)

```
ParseResult parseLine(String line, {String separator = defaultSeparator}) {
  final parts = line.split(separator).map((part) => part.trim()).toList();
  if (parts.length != fieldCount) {
    return (
      lesson: null,
      error: 'очікується $fieldCount полів, знайдено ${parts.length}',
    );
  }
  ...
  final weekday = int.tryParse(parts[_weekdayIndex]);
  if (weekday == null) {
    return (
      lesson: null,
      error: 'день тижня «${parts[_weekdayIndex]}» не є числом',
    );
  }
  ...
}
```

`DateTime.tryParse` не відхиляє неіснуючі дати: рядок `2026-13-26` він перетворює на 26 січня 2027 року. Тому `parseDate` перетворює результат назад у рядок і порівнює з вхідним; якщо вони різні, дата некоректна (лістинг 3).

Лістинг 3 – Перевірка дати

```
DateTime? parseDate(String raw) {
  final date = DateTime.tryParse(raw);
```

```
  if (date == null) return null;
  final normalized = date.toIso8601String().substring(0, isoDateLength);
  return normalized == raw ? date : null;
}
```

Функція `validateLesson` перевіряє вже розібране заняття й повертає список порушень (порожній, якщо їх немає). Перевіряються тип заняття, день тижня (1–7), належність часу до початків шести пар, формат аудиторії (`DL` або літера корпусу з трьома цифрами), формат прізвища викладача та збіг дня тижня з днем тижня дати першого заняття (лістинг 4). Остання перевірка відсіює суперечливі записи, які поодинці виглядають коректними.

Лістинг 4 – Фрагмент валідатора

```
List<String> validateLesson(Lesson lesson) {
  final problems = <String>[];
  ...
  if (!pairStartMinutes.contains(lesson.startMinutes)) {
    problems.add('час ${formatTime(lesson.startMinutes)} не є початком
пари');
  }
  ...
  if (!teacherPattern.hasMatch(lesson.teacher)) {
    problems.add(
      'викладач «${lesson.teacher}» має мати формат «Прізвище І. Б.»',
    );
  }
  if (lesson.firstDate.weekday != lesson.weekday) {
    problems.add('день тижня не збігається з днем тижня дати');
  }
  return problems;
}
```

1.2.4 Замикання

Функція `makeCounter` повертає функцію, яка захоплює локальну змінну `count` (лістинг 5). Ця змінна не зникає після завершення `makeCounter`: вона зберігається в замиканні, тому кожен виклик повернутої функції збільшує те саме значення. Кожен виклик `makeCounter` створює власну змінну `count`, тож лічильники незалежні. У програмі лічильник нумерує записи, що не пройшли перевірку. Фабрика `onWeekday` захоплює параметр `weekday` і повертає умову для `where`; так само працює `ofType` із параметром `type`.

Лістинг 5 – Замикання

```
int Function() makeCounter() {
  var count = 0;
  return () => ++count;
}

Predicate<Lesson> onWeekday(int weekday) =>
    (lesson) => lesson.weekday == weekday;
```

1.2.5 Конвеєри й групування

Фільтрацію виконує `filterLessons` з умовами `onWeekday` і `ofType`, перетворення заняття на рядок – `toTimetableLines`. Згортання реалізовано двічі: `startTimeStats` обчислює найраніший, найпізніший і середній час початку одним `fold` із записом-акумулятором, а `averageStart` будує ланцюжок `where → map → fold` (лістинг 6).

Групування виконує узагальнена функція `groupBy`, що повертає `Map<K, List<T>>`; для викладачів це `Map<String, List<Lesson>>`, а кількість занять у групі рахує `countByTeacher`. Накопичення в `groupBy` зроблено циклом, бо тут елементи складаються в `Map`, а не обчислюється одне значення.

Лістинг 6 – Згортання та групування

```
double? averageStart(Iterable<Lesson> items, Predicate<Lesson> test) {
  final minutes = items.where(test).map((lesson) => lesson.startMinutes);
  if (minutes.isEmpty) return null;
  return minutes.fold<int>(0, (sum, value) => sum + value) /
minutes.length;
}

Map<K, List<T>> groupBy<K, T>(Iterable<T> items, K Function(T) keyOf) {
  final result = <K, List<T>>{};
  for (final item in items) {
    (result[keyOf(item)] ??= []).add(item);
  }
  return result;
}
```

1.2.6 Сортування

Функція `sortLessons` приймає порівняння за іменованим параметром `by` і прапорець `descending` (лістинг 7). Є два критерії: `byTime` (день тижня, потім час початку) та `bySubject` (назва дисципліни, потім час). Вхідний список не змінюється,

повертається відсортована копія. Зворотний порядок розвертає весь список, тому дні й заняття всередині дня йдуть у зворотному порядку.

Лістинг 7 – Сортування

```
List<Lesson> sortLessons(
  List<Lesson> items, {
  LessonComparator by = byTime,
  bool descending = false,
}) {
  final sorted = [...items]..sort(by);
  return descending ? sorted.reversed.toList() : sorted;
}
```

1.2.7 Регулярний вираз і маскування

Формат прізвища викладача перевіряє регулярний вираз `teacherPattern` з іменованими групами `surname`, `first` і `second`, записаний сирим рядком (лістинг 8). Функція `maskTeacher` викликає `replaceAllMapped` і залишає видимою лише першу літеру прізвища: «Баличев М. Д.» стає «Б****** М. Д.». Колбек `replaceAllMapped` отримує загальний тип `Match`, а іменовані групи є лише в `RegExpMatch`, тому збіг приводиться до цього типу. Для `RegExp` таке приведення безпечне. Оператор `!`, ключове слово `late` і тип `dynamic` у коді не вживаються.

Лістинг 8 – Регулярний вираз і маскування

```
final teacherPattern = RegExp(
  r'^(?<surname>[А-ЯІЇЄҐ][а-яіїєґ’-]+) '
  r'(?<first>[А-ЯІЇЄҐ])\. (?<second>[А-ЯІЇЄҐ])\.$',
);

String maskTeacher(String teacher) =>
    teacher.replaceAllMapped(teacherPattern, (match) {
      final groups = match as RegExpMatch;
      final surname = groups.namedGroup('surname') ?? '';
      final visible = surname.substring(0, visibleSurnameChars);
      final hidden = maskChar * (surname.length - visibleSurnameChars);
      return '$visible$hidden ${groups.namedGroup('first')}. '
          '${groups.namedGroup('second')}.';
    });
```

1.2.8 Дати та звіт

Вік запису обчислює `timeSinceFirst`: різниця `DateTime.now()` і дати першого заняття має тип `Duration`, у виводі використано `inDays`. Функція `buildReport`

формує звіт через `StringBuffer`; її поведінку задають іменовані параметри `topCount`, `descending`, `separator` і `maskTeachers` зі значеннями за замовчуванням (лістинг 9). Програма виводить звіт двічі: звичайний і з масками прізвищ, зворотним порядком днів та іншим роздільником.

Лістинг 9 – Вік запису та сигнатура звіту

```
Duration timeSinceFirst(Lesson lesson, {DateTime? now}) =>
    (now ?? DateTime.now()).difference(lesson.firstDate);

String buildReport(
  List<Lesson> items, {
  int topCount = defaultTopCount,
  bool descending = false,
  String separator = defaultColumnSeparator,
  bool maskTeachers = false,
}) {
  ...
}
```

1.3 Результати виконання

Аналізатор не виявив зауважень (рисунок 1), команда `dart format` не змінює файлів. Програма обробила 22 рядки й визнала коректними 15. Те, як оброблено тестові записи, показано в таблиці 2 і на рисунку 2.

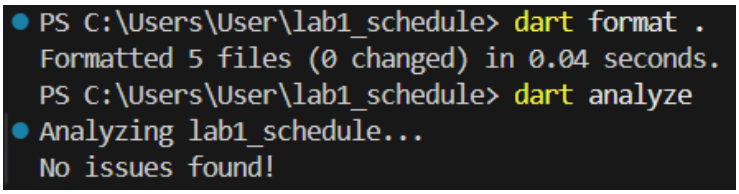


Рисунок 1 – Результат виконання dart analyze

Таблиця 2 – Обробка тестових записів

Запис	Дефект у даних	Повідомлення програми
16	Немає поля аудиторії	Не розібрано: очікується 7 полів, знайдено 6
17	День тижня «субота»	Не розібрано: день тижня не є числом
18	Дата 2026-13-26	Не розібрано: некоректна дата
19	Час 10:00	Час не є початком пари
20	День тижня 8	День поза діапазоном 1–7; день не збігається з днем дати

Запис	Дефект у даних	Повідомлення програми
21	Викладач «шевченко»	Викладач має мати формат «Прізвище І. Б.»
22	День 2, дата в середу	День тижня не збігається з днем тижня дати

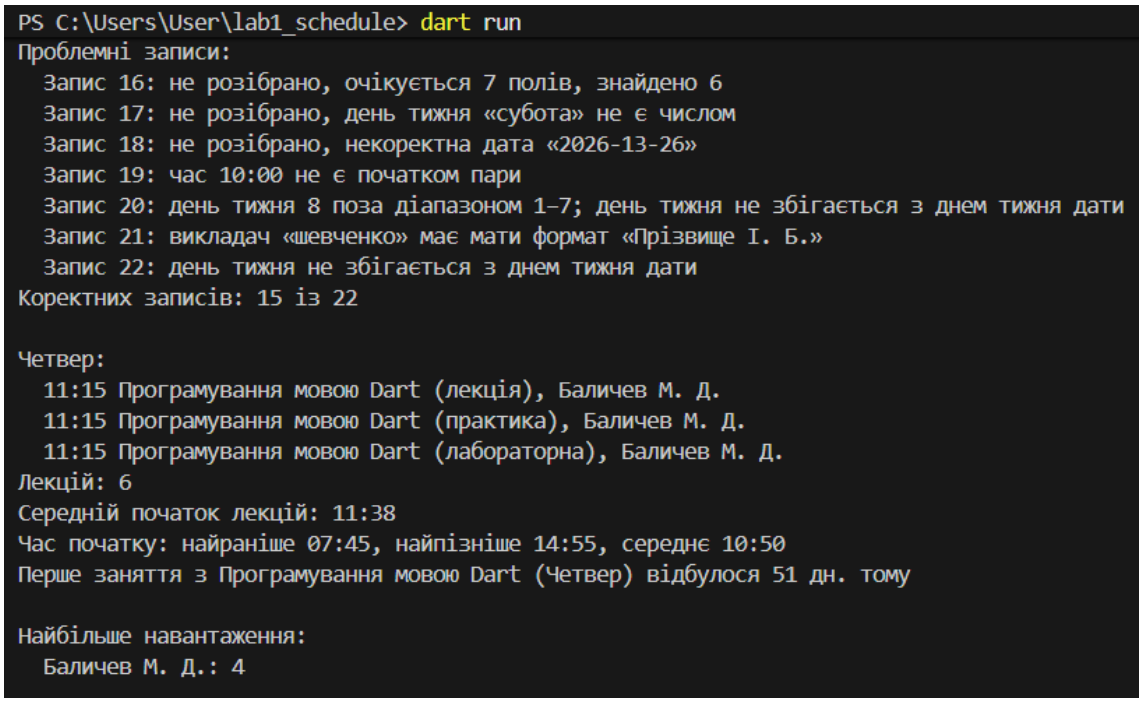


Рисунок 2 – Вивід програми: проблемні записи та статистика

Шість коректних записів мають тип «лекція» (по одному для чотирьох дисциплін і два для Dart), середній початок цих лекцій – об 11:38; найраніше заняття починається о 07:45, найпізніше – о 14:55. Найбільше навантаження має викладач Баличев М. Д. (чотири заняття). Другий звіт показує ті самі заняття із замаскованими прізвищами (рисунок 3).

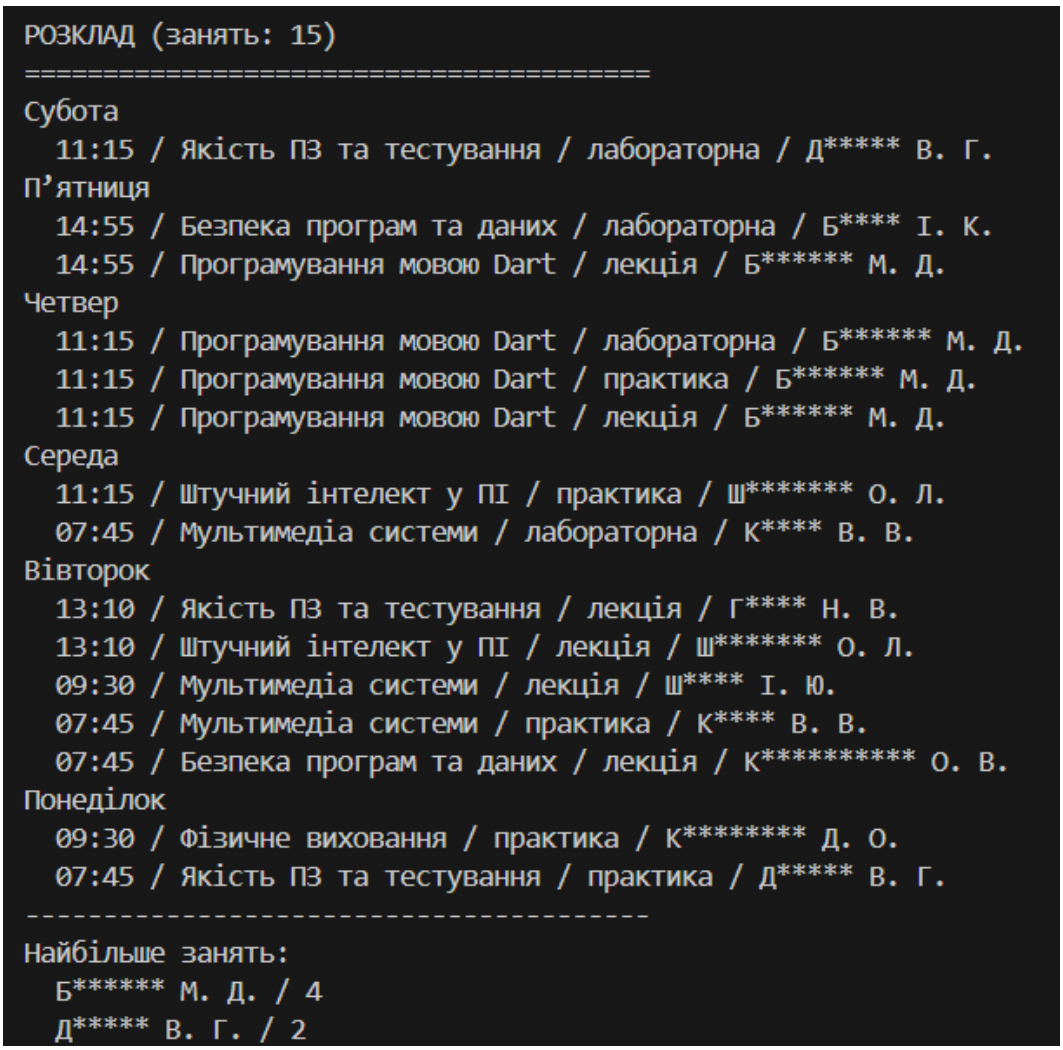


Рисунок 3 – Вивід програми: звіт із масками прізвищ

1.4 Висновки

У роботі реалізовано консольну програму для обробки розкладу занять групи на Dart без використання класів. Дані розбираються й перевіряються без винятків: помилка вхідного рядка повертається як значення запису, що прибирає потребу в `try/catch`. Виявлено, що `DateTime.tryParse` не відхиляє неіснуючі дати, тому для них потрібна додаткова перевірка.

Опановано замикання, що зберігають стан між викликами, конвеєри `where`, `map` і `fold`, групування в `Map`, сортування за кількома критеріями, регулярні вирази з іменованими групами та арифметику дат через `Duration`. Записи (records) виявилися зручнішими за `Map` для типізованих даних. Винесення логіки в `lib/` готує проєкт до модульних тестів у наступних роботах. Код не має зауважень `dart analyze` і

відформатований `dart format`, історія змін у Git налічує понад п’ять комітів, зроблених у різні дні.

1.5 Перелік використаних джерел

1. Dart Team. Dart language documentation. URL: https://dart.dev/language (дата звернення: 29.10.2026-01.11.2026).
2. Dart Team. Effective Dart. URL: https://dart.dev/effective-dart (дата звернення: 29.10.2026-01.11.2026).
3. Dart Team. Records. URL: https://dart.dev/language/records (дата звернення: 29.10.2026-01.11.2026).
4. Dart Team. Functions. URL: https://dart.dev/language/functions (дата звернення: 29.10.2026-01.11.2026).
5. Dart Team. Patterns. URL: https://dart.dev/language/patterns (дата звернення: 29.10.2026-01.11.2026).
6. Dart Team. Customizing static analysis. URL: https://dart.dev/tools/analysis (дата звернення: 29.10.2026-01.11.2026).
7. Dart Team. API reference: dart:core. URL: https://api.dart.dev/ (дата звернення: 29.10.2026-01.11.2026).
8. Dart Team. Collections. URL: https://dart.dev/language/collections (дата звернення: 29.10.2026-01.11.2026).
9. Dart Team. Iterable collections. URL: https://dart.dev/libraries/collections/iterables (дата звернення: 29.10.2026-01.11.2026).